

Case Study: Building a Structured Risk Intelligence pipeline

The following case study is intended to measure your ability to understand, analyse, solve and advise on a business problem. We will use it to assess how you approach the problem, propose and construct a reasonable solution and layout a strategy for deployment in production including potential future improvements. We are interested in discussing the proposed solution at a high-level from a stakeholder engagement point of view, as well as at a deep technical level.

As time for completing the task is very limited (~4–8 hours), we suggest that all attempts (even unsuccessful ones) are kept for discussion. We are interested in your train-of-thought process, how that is put into use in your technical solution and how you think about building a real product, more than in the level of performance of the solution itself.

We expect you to use AI coding assistants (Cursor, Copilot, Claude Code, your IDE's built-in assistant, whatever you normally use). The walkthrough afterwards is where we'll calibrate your understanding, so reason as you build.

The product you're joining

You're joining a small team at a research firm that sells **structured risk intelligence** to asset **managers and compliance teams**. Today our analysts read 200+ corporate annual reports per quarter to track each company's principal risks: how they're **disclosed**, how **categorization** shifts year-over-year and which **emerging risks** appear across a portfolio. The work is **slow, expensive and inconsistent**: two analysts reading the same report often surface different risks.

We're building a **structured, queryable risk database** that turns those reports into something analysts and clients can search, filter and trend over time. Example questions the database should be able to answer:

- *"What are the top enterprise risks facing Vestas, and how frequently are they reviewed at Board/Executive level?"*
- *"What emerging risks have been newly elevated?"*
- *"Show me every renewable-energy company that lists cyber security as a principal risk"*

Your job in this case: build the pipeline that turns one report into structured data and do it well.

The task

We're giving you **Vestas's Annual Report** (PDF, ~190 pages). Build a **small pipeline** in python that:

1. Ingests the PDF.
2. **Extracts the company's principal risks**. For each risk, produce:
 - a. A short title
 - b. A 2-3 sentence description
 - c. A category (e.g. financial, operational, regulatory, market, climate, cyber, supply chain)
 - d. The section and page where you found it
 - e. Any stated mitigation action

3. Returns the result as a **structured object** (JSON, Pydantic, your choice of schema language).
4. **Evaluates** itself. Build a small eval (a golden set + at least one evaluator) that would catch a **regression**, for example by swapping in a weaker model, breaking the parser, or changing a prompt.

You may choose to structure your pipeline as a single step or multiple steps/tools. We are interested in how you decompose the problem and why (e.g. parsing vs extraction, single-pass vs multi-step). Considering the time available for the case, implement core part of the solution (your choice) while ready to provide a walkthrough of how the technical solution will be built into a real product.

Scope

Focus on these sections of the PDF:

- Risk management (p.50-51)
- Material impacts, risks, and opportunities (p.71-74)

You may also pull from Cyber Security (p.118) and Climate Change (p.85-92). You decide what parts of the PDF to ingest and you do not need to process all 190 pages.

Think product, not just task

Before you write code, write a short PLAN.md (~½ page). Situate this single-report pipeline inside the broader product:

- Who is the user, and how will they consume the output?
- Roughly what does the product surface look like (batch API, interactive UI, alerting, ...)?
- What are you optimizing for in this first slice: correctness, coverage, cost, throughput?
- What assumptions are you making?
- What are you explicitly *not* solving here but would tackle next?

This isn't a polished product spec. It's a ~10-minute think-through that should visibly inform every implementation decision you make next.

Deliverables

- PLAN.md: product context, user, optimization target, assumptions, what's deferred.
- Working code: any stack, any model provider. Be ready to explain why you chose what you chose.
- DESIGN.md (~1 page): your stack, parsing approach, your decomposition of the work, and key trade-offs. The trade-offs should trace back to choices in PLAN.md. Include considerations for scaling (latency, cost or throughput) if this were run across many reports
- Eval artifacts: your golden set, the evaluator(s), and at least one sample run catching a realistic failure mode
- STRETCH.md (short): what you'd build next if you had another week.

Walkthrough format (45 min total)

After we receive your submission, we'll schedule a 60-minute walkthrough.

- 30 minutes are yours. You drive. Prepare a focused walkthrough of the parts of your solution that *you* think matter most. You won't have time to cover everything and deliberately choosing what to emphasize (and what to skip) is part of what we're listening for. Lead us through your reasoning, not just your code. We'll stay quiet unless we need a quick clarification.
- 20 minutes for Q&A. We'll follow up on what you showed, probe areas you skipped and depending on role level, discuss how you'd evolve the solution at scale.
- 10 minutes for your Questions. The final few minutes are yours for questions to us.

Constraints & ground rules

- Time budget: ~4–8 hours of focused work. We assess what you got done *and* how you reasoned about your cuts.
- AI coding tools are allowed and expected. The walkthrough will probe understanding deeply enough that copy-paste alone won't pass.
- No stack adherence. You're not graded on framework or provider choice. You're graded on outcomes and reasoning.
- No starter code. You choose the project layout, dependencies, and structure. These choices are part of the task and what we evaluate.
- Hybrid approaches welcome. Combining classical NLP (NER, regex, sentence segmentation, traditional information extraction) with LLM calls is fully encouraged. We're hiring for judgment about *when* to reach for an LLM, not for LLM mastery alone.

Practicalities

- Use Vestas Annual Report PDF shared with you
- Submission: Please send a zip or git repo link
- Questions during the take-home: If anything in this brief is genuinely blocking you, reach out. We'd rather you ask than be stuck.

Good luck. We're looking forward to seeing how you think.